Potential Method Proof

Proof:

$T := \sum_i^n t_i$ time taken by all n operations

Want to show $T \leq n f(n)$ amortized time * n

$nf(n) = \sum_{i=1}^n f(n)$ telescope

$\geq \sum_{i=1}^n [t_i + (\Phi(s_i) - \Phi(s_{i-1}))]$

Given:
$t_i + [\Phi(s_i) - \Phi(s_{i-1})] \leq f(n)$

$nf(n)$

$\geq [t_1 + \Phi(s_1) - \Phi(s_0)] + [t_2 + \Phi(s_2) - \Phi(s_1)] + (t_3 + \Phi(s_3) - \Phi(s_2)] \ldots + (t_n + \Phi(s_n) - \Phi(s_{n-1}))$

$\geq t_1 + t_2 + \ldots + t_n + \Phi(s_n) - \Phi(s_0)$

$\geq T + \Phi(s_n) - \Phi(s_0)$

$\geq T + 0$

$\geq T$

Given
$\Phi(s_i) \geq \Phi(s_0)$
$\Phi(s_i) - \Phi(s_0) \geq 0$

$nf(n) \geq T$

□ Shows that the total time is less than nf(n)

Splay trees

- one of the most interesting balanced BST algorithms
- amortized log(n)
- worst case is arbitrary bad

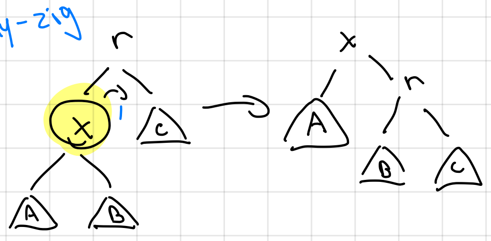
- after you find a node in a tree, pull that node up the tree to make work helpful
- pull to root to make that work useful

these operations:
- get amortized time bound

```
def splay(node):
    while node is not root:
        if node is zig from root:
            do splay-zig rotation to move node to the root level
        if node is zig-zig from grandparent:
            do splay-zig-zig rotation to move node up two levels
        if node is zig-zag from grandparent:
            do splay-zig-zag rotation to move node up two levels
```

3 cases:

splay-zig



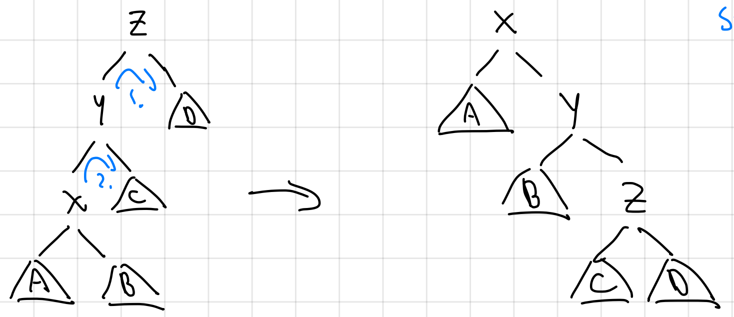
steps
1. 1 rotation

1 splay node that is 1 step off from the root

---

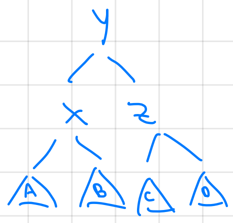
splay zig-zig

left-left
right-right
from grandparent

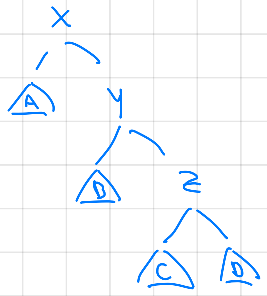


steps
1. rotate around grandparent
2. rotate around parent

1.



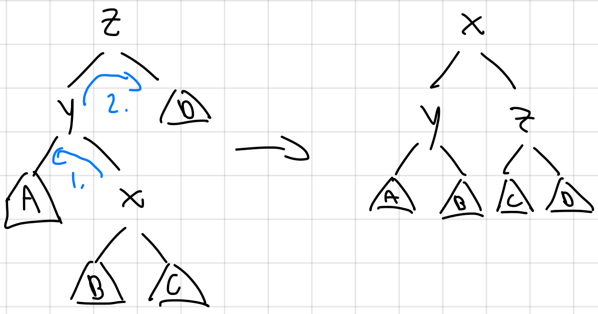
2.



---

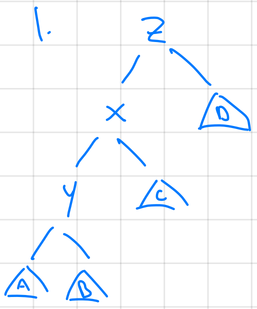
splay zig-zag

left right
right left
from grandparent

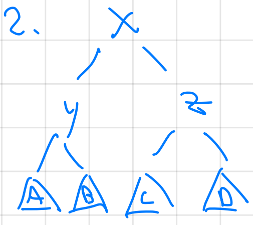


steps.
1. rotate around parent
2. rotate around grandparent

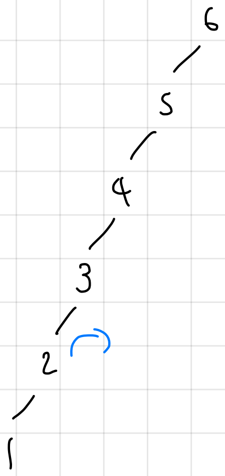
1.



2.



splay node 1

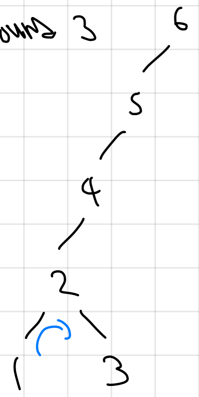


we will use a bottom up splay tree

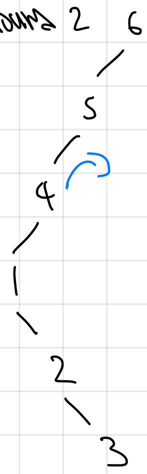
Only look at target and next 2 layers

1. splay zig-zig

around 3

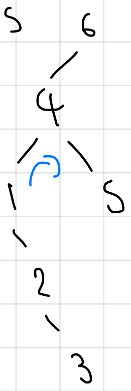


around 2

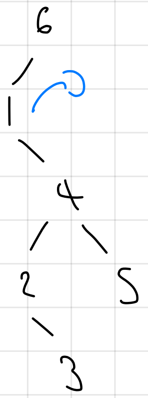


2. splay zig-zig

around 5

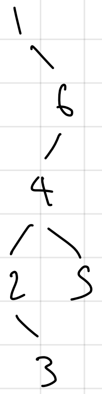


around 4



3. splay zig

around 6



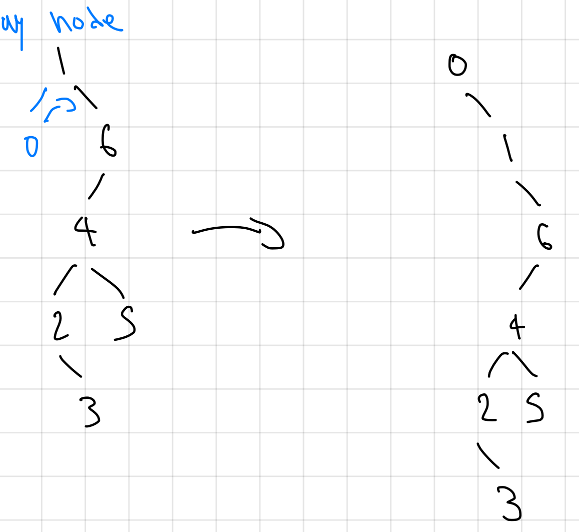
splay trees
- do the normal BST function
- splay node

```
def splay_insert(root, v):
    new_node = normal_bst_insert(root, v)
    splay(new_node)
    return new_node
```

```
def splay_search(root, v):
    if root is empty:
        return not found
    if v == root.value:
        splay(root)
        return root
    child ← root.left if v < root.value else root.right
    if child is empty:
        splay(root)
        return not found
    return splay_search(child, v)
```

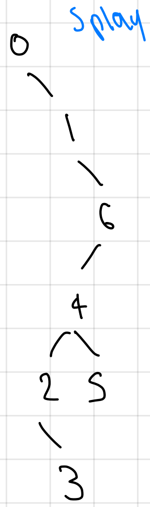
Insert 0

1. normal BST 0
2. splay node



Search 3.5 − still splay if not found

Splay 3



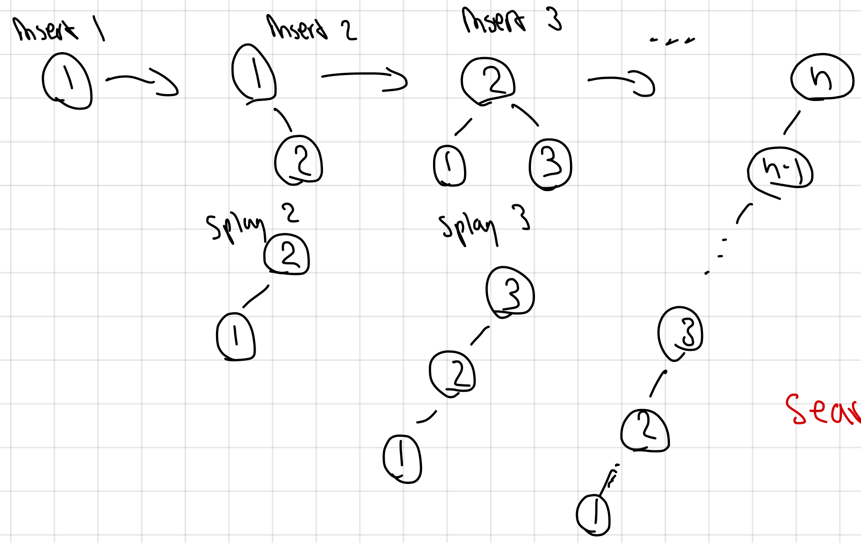
---

worst-case per operation for search
- linked list
- always search the deepest leaf

Insert 1
Insert 2
⋮
Insert n

} in order to make a linked list

search 1 O(n) all the way to the bottom



Search 1 O(n)

however, all of the insertions are O(1), since it is next to the root

after the O(n) search, the node is splayed and the repeated searches are O(1)

amortized time

O(log(n))

as the number of operations increase, it gets more balanced since it isn't a linear linked list

worse case:

O(n)

very difficult to have O(n) operations in a row

---

worst case time to search for the same time n times

worst case: first search : O(n)
n-1 searches : O(1)

amortized : $\frac{\text{total time}}{n} = \frac{n + (n-1)(1)}{n} = \frac{2n-1}{n} \approx 2$ O(1)

why splay when a search fails

In the worst case, the search takes $O(n)$

If we don't splay, repeated failed search will still be $O(n)$

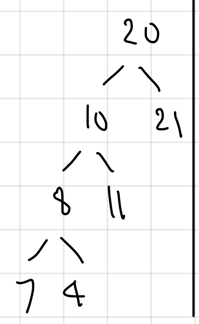
no longer amortized $O(\log(n))$

---

operations easy on splay trees, but hard on other trees

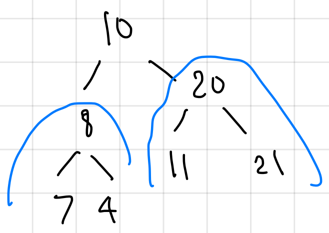
split tree into 1 tree with everything greater than v and 1 tree with everything less than v

Given:



Split at 10

1. splay at 10



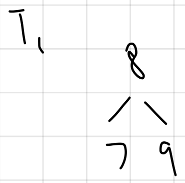
2. return left child
disconnect children
return right child

---

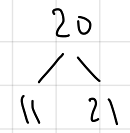
join two trees T_1 and T_2

all elements in T_2 are greater than all elements in T_1

T_1



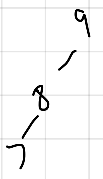
T_2



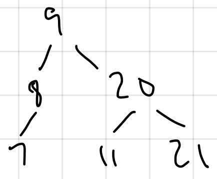
1. find max in T_1 - splay max in T_1
   - splayed max will have no right child

2. put T_2 as the right child

splay 9



make T_1 the right child



---

delete

- split at deleted node
- join the two splits

---

amortized time

Given: BST of $\leq n$ nodes

do at least n operations splay tree

it will be $O(\log(n))$ amortized time per operation

non uniform searches have better time

splay trees are as good as every other tree